

StockAPP

Especificación Técnica Integral y Biblioteca de APIs
DOCUMENTACIÓN OFICIAL DE ARQUITECTURA — V1.1.0

	Sistema:
StockAPP (Gestión de Stock y Punto de Venta)	
	Topología:
Microservicios Desacoplados (Go + Express)	
	Red y Túneles:
Tailscale WireGuard P2P + Bridge Podman	
	Seguridad:
2FA TOTP (RFC 6238) + JWT HS256 + bcrypt	
	Base de Datos:
PostgreSQL 16 (Multi-Tenant por id_negocio)	
	Catálogo Maestro:
21.800+ Productos Argentinos EAN-13	

1. Visión General y Alcance del Sistema

StockAPP es una solución moderna de software como servicio (SaaS) orientada a comercios minoristas (kioscos, minimercados, almacenes). Su arquitectura fue concebida para brindar tres beneficios esenciales:

- Ultra bajo consumo de memoria y respuesta instantánea:** La aplicación de mostrador utiliza Tauri v2 y Svelte 5, requiriendo menos de 90 MB de RAM y respondiendo a lecturas de código de barras en menos de 10 milisegundos.
- Cero configuración de red para el cliente:** Cada comercio se comunica con la nube a través de un túnel punto a punto seguro (Tailscale / WireGuard), eliminando la necesidad de abrir puertos o configurar IP fija en los routers de los comercios (MikroTik, etc.).
- Aislamiento estricto de inquilinos (Multi-tenant):** Cada transacción y consulta de stock está aislada lógicamente por `id_negocio`, garantizando que un negocio jamás pueda visualizar o modificar información de otro comercio.

2. Arquitectura de Microservicios y Topología de Red

La plataforma separa nítidamente las responsabilidades en microservicios especializados conectados en una red interna privada:

CLIENTE LOCAL (DESKTOP) Tauri v2 + Svelte 5 (:1420) Sidecar Go (tsnet :9090) Túnel P2P

SERVIDOR EN LA NUBE (RED BRIDGE PRIVADA) stock-api (:3000) Node 22 / Express 5 Auth, JWT,

2FA y Proxy stock-operations (:8080) Go 1.25 ServeMux Ventas, Stock y Facturación /session

PostgreSQL 16 (:5432) Puerto Cerrado al Exterior Multi-Tenant por id_negocio front-admin

(:5173) Vue 3 + Nginx Acceso por Tailscale

FIGURA 1: TOPOLOGÍA DE MICROSERVICIOS Y FLUJO DE PAQUETES

3. Modelo de Comunicaciones Inter-Servicios

App Tauri Sidecar Go stock-api stock-operations 1. POST /api/ventas (local:9090) 2. Túnel P2P

Cifrado (tsnet) Valida JWT 3. Proxy HTTP Upstream (:8080) 4. GET /internal/session Claims en

Cache 5. 201 Created (Venta registrada y stock descontado)

FIGURA 2: CICLO COMPLETO DE UNA TRANSACCIÓN EN EL SISTEMA

La comunicación entre los componentes se realiza siguiendo contratos estrictos:

- **Desacoplamiento Criptográfico:** `stock-operations` nunca conoce el `JWT_SECRET`. Valida la validez de los tokens delegando en `stock-api` mediante introspección interna y conservando las respuestas en caché de memoria durante 60 segundos.
- **Defensa en Capas para la Administración:** Las llamadas a `/api/admin/*` son filtradas en el gateway (comprobando que el claim sea `rol == 'superadmin'`) y reforzadas en Go con el middleware `conSuperAdminAuth`.

4. Modelo de Datos y Base de Datos (PostgreSQL 16)



FIGURA 3: DIAGRAMA ENTIDAD-RELACIÓN (ERD) PRINCIPAL DE POSTGRESQL

5. Biblioteca Exhaustiva de APIs (API Catalog & Reference)

A continuación se documentan todos los endpoints disponibles en el sistema con sus contratos de datos, cabeceras requeridas, métodos y respuestas de estado.

Módulo: Autenticación y Terminales (`stock-api`)

POST `/api/auth/login` **Público**

Inicia sesión para Dueños y Cajeros. Emite el token JWT (15 min) y el refresh token (30 días).

Body JSON:

```
{ "usuario": "carlos", "password": "clave123", "dispositivo": "Caja 1" }
```

Respuesta 200 OK:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2Vybm91dCI6ImNhclosIiwiaWF0IjoxNjU0MjY0MDA0LCJleHAiOjE2NTQyNjQ0MDA0fQ.",
  "refresh_token": "a8f9c...",
  "usuario": { "id_usuario": 14, "id_negocio": 3, "nombre": "Carlos", "rol": "dueño" }
}
```

POST **/api/auth/refresh** **Público**

Rota el token de refresco vencido emitiendo un nuevo par de tokens.

Body JSON:

```
{ "refresh_token": "a8f9c..." }
```

Respuesta 200 OK:

```
{ "access_token": "eyJ...", "refresh_token": "nuevo_hash..." }
```

POST **/api/auth/logout** Público

Invalida el token en la base de datos eliminando la sesión del comercio.

Respuesta: 204 No Content

POST **/api/auth/obtener-tailscale-key** Público (Valida Dueño)

Entrega una clave de conexión P2P para incorporar una computadora a la red del negocio (máximo 4

terminales por comercio).

Body JSON:

```
{ "email": "duenio@kiosco.com", "password": "...", "device_id": "dev-uuid", "nombre_dispositivo": "Terminal 2" }
```

Respuesta 200 OK:

```
{ "ts_auth_key": "tskey-auth-...", "dispositivos_activos": 2, "max_dispositivos": 4 }
```

Módulo: SuperAdministrador y 2FA (stock-api)

POST **/api/admin/login** Público

Paso 1 de acceso global. Valida credenciales con bcrypt y entrega el Data URL del código QR o el desafío de 6

dígitos.

Body JSON:

```
{ "usuario": "admin", "password": "..."} 
```

Respuesta 200 OK:

```
{ "requiere_2fa": true, "setup_totp": false, "temp_token": "eyJhb..." } 
```

POST **/api/admin/totp/verificar** Requiere temp_token

Paso 2 de acceso global. Valida los 6 dígitos del celular (tolerancia $\pm 30s$) y emite el JWT con rol [superadmin](#).

Body JSON:

```
{ "temp_token": "eyJ...", "codigo": "839201"} 
```

Respuesta 200 OK:

```
{ "token": "jwt_superadmin...", "admin": { "id_admin": 1, "usuario": "admin" } } 
```

Módulo: Catálogo, Stock e Inventario (stock-operations)

GET /api/productos?buscar={query}&limite={n} Cajero / Dueño

Búsqueda inteligente en los 21.800+ productos argentinos y en el inventario propio.

Respuesta 200 OK:

```
"en_stock": true,
```



GET **/api/stock** Cajero / Dueño

Devuelve todos los productos que el comercio tiene cargados en inventario con sus precios vigentes.

Respuesta 200 OK: Array con lista de existencias, precio de costo y stock mínimo.

PUT **/api/stock/{id_producto}** Solo Dueño

Actualiza existencias y precios de venta de un producto.

Body JSON:

```
{ "cantidad_disponible": 45, "precio_venta": 1600.0, "precio_costo": 1050.0, "stock_minimo": 10 }
```

Respuesta 200 OK:

```
{ "mensaje": "stock actualizado correctamente" }
```

Módulo: Ventas y Caja (stock-operations)

POST **/api/ventas** Cajero / Dueño

Registra una venta atómica: valida precios en el servidor, comprueba stock disponible y descuenta las

existencias.

Body JSON:

```
{
  "id_venta": 512,
  "id_cliente": 1,
  "id_usuario": 1,
  "metodo_pago": "efectivo",
  "fecha_hora": "2026-09-08 16:20:00",
  "items": [
    {
      "id_producto": "7790895000997",
      "cantidad": 2
    }
  ],
  "total": 3000.0
}
```

Respuesta 201 Created:

```
{
  "id_venta": 512,
  "id_cliente": 1,
  "id_usuario": 1,
  "metodo_pago": "efectivo",
  "fecha_hora": "2026-09-08 16:20:00",
  "items": [
    {
      "id_producto": "7790895000997",
      "cantidad": 2
    }
  ],
  "total": 3000.0
}
```

```
{
  "metodo_pago": "efectivo",
  "items": [
    {
      "descripcion": "Coca Cola 500ml",
      "cantidad": 2,
      "precio_unitario": 1500.0,
      "subtotal": 3000.0
    }
  ]
}
```

GET **/api/resumen** Solo Dueño

Devuelve el balance en tiempo real del turno o fecha actual con discriminación por medio de pago.

Respuesta 200 OK:

```
{
  "total_ventas": 98500.0,
  "cantidad_operaciones": 34,
  "desglose_pagos": {
    "efectivo": 60000.0,
    "transferencia": 25000.0,
    "tarjeta": 13500.0
  }
}
```

